Web Programming Course End Project Report On

College Event Management Website

in

COMPUTER SCIENCE AND ENGINEERING

(IoT with Cyber Security Including Blockchain Technology)

by

Names with
Roll numbers

Under the Supervision of

N. Sujata Gupta

Assistant Professor

Department of Computer Engineering and Technology



Department of Computer Engineering and Technology

Chaitanya Bharathi Institute of Technology

(Autonomous)

(Affiliated to Osmania University, Hyderabad)

Hyderabad, TELANGANA (INDIA) - 500 075

[2025-2026]



CERTIFICATE

This is to certify that the Course End Project titled **"College Event Management Website"** is a bonafide work carried out by the students named on the title page, of the Department of Computer Engineering and Technology, Chaitanya Bharathi Institute of Technology, Hyderabad, Telangana (India), submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Engineering in the academic year 2025-2026.

Coordinators

Ms. N. Sujata Gupta
Assistant Professor, Department of CET, CBIT

Place: Hyderabad, Telangana
Date: _____

Head of the Department
Professor, CET Department, CBIT

Table of Contents

Section	Page
Title Page	i
Certificate	ii
Index (Table of Contents)	iii-iv
List of Figures	v
List of Tables	vi
Abstract	vii
1. Introduction	1
1.1 Background	1
1.2 Motivation	1
1.3 Project Scope	2
1.4 Technologies Used	2
2. Problem Statement	3
2.1 Limitations of Existing Systems	3
2.2 Need for an Integrated Event and Club Platform	3
2.3 Importance of Data Privacy and User Experience	4
2.4 Lack of Personalization and Adaptability	4
2.5 Integration and Scalability Issues	4
2.6 Use Case Overview	5

Table of Contents (continued)

Section	Page
3. Literature Review	6
3.1 Evolution of Digital Event Management Systems	6
3.2 Adaptive and Personalized Event Management Models	6
3.3 Data Visualization and Insight Generation Systems	6
3.4 Integration of Security Features in Digital Applications	6
3.5 Comparison to Existing Tools	7
3.6 Research Gaps	7
3.7 Conceptual Framework Model	8
4. Proposed Model	9
4.1 System Overview	9
4.2 System Architecture	10
4.3 System File Structure	11
4.4 Key System Components	13

List of Figures

Figure	Page
Figure 1. UML use case overview	5
Figure 2. Conceptual framework model	8
Figure 3. System architecture and technology layers	10

List of Tables

Table	Page
Table 1. Comparison of event-management approaches	7

Abstract

College events are often announced through notices, messages and separate registration forms. This arrangement makes it difficult for students to identify relevant activities and for organizers to maintain consistent participant and fee records. The College Event Management Website, implemented as Campus Commons, brings these activities into a common platform for students, clubs and administrators.

Students can create an account, maintain a profile, browse clubs and events, join clubs, register for activities and review their registrations. Clubs maintain organizational profiles and create events with specified dates, venues, fees, capacities and eligibility conditions. Administrators review club applications, inspect user records and moderate events. Each role receives an interface suited to its responsibilities.

The frontend uses React, TypeScript, Vite and Tailwind CSS. A Java Spring Boot backend provides application services, applies Spring Security access controls and manages records through Spring Data JPA. PostgreSQL is the main database, with Supabase PostgreSQL available as a compatible hosting option. The frontend presents information and collects input, while the backend validates requests and maintains the shared records.

The platform supports both free and fee-based events. Free registrations are confirmed directly. The current fee-payment facility is a development simulator that demonstrates payment status and cancellation behaviour without collecting real money. Public event information is separated from personal records and administrative controls.

This report reviews the project background, problem statement, relevant approaches and proposed model. It explains the principal components, conceptual framework, architecture and source organization at an overview level. The result is a coordinated design for college event administration that gives students clear access to activities and gives organizers a common place to manage participation.

Keywords: college event management, clubs, React, Java, Spring Boot, PostgreSQL.

1. Introduction

A college event involves more than publishing a title and date. The organizer must communicate eligibility, reserve a venue, control participation and account for registration fees. Students need an accurate description of the activity and a reliable indication that their booking has been accepted. The proposed website connects these responsibilities through a shared record of clubs, events and registrations.

1.1 Background

Campus activities include technical workshops, cultural programmes, sports meetings and student-club sessions. Their administration commonly involves a mixture of announcements, forms and spreadsheets. When the same event is represented in several places, a revised deadline or venue may not reach every participant. A centralized website can make the published event record the reference point for students and organizers.

The project uses a browser interface backed by a Java application service. Public information remains easy to discover, while personal registrations and organizer controls require authentication. The database stores durable records so that a page refresh or a change of browser does not remove a student's booking history.

1.2 Motivation

The primary motivation is to reduce manual coordination while preserving clear accountability. Students should be able to distinguish an available event from a full or closed event before attempting registration. Clubs should be able to publish complete information and review participants without maintaining duplicate lists. Administrators need a way to review organizations and respond to inappropriate or cancelled events.

The project also demonstrates how a responsive frontend can work with a structured backend. Clear forms make the system approachable, while common records and server-side checks make its results dependable. Both aspects are needed for a practical college platform.

1.3 Project Scope

The scope covers one college and three roles: student, club and administrator. It includes account access, club membership, event publication, registration and a simulated fee-payment facility. Event eligibility is evaluated against the department and study year entered in a student profile.

• **Students:** register or log in, browse clubs and events, join clubs, register for events, complete the available fee step and view registration history.

• **Clubs:** register or log in, maintain a profile, create and edit events, publish approved activities and manage student registrations.

• **Administrators:** review users and clubs, approve or reject club applications, and moderate event publication.

The scope is limited to the basic functions described above. Live financial settlement and institutional identity verification are outside the current implemented scope.

1.4 Technologies Used

React and TypeScript implement reusable pages, typed information models and interactive forms. **Vite** supports frontend development and building the browser application. **Tailwind CSS** provides styling utilities for consistent, responsive layouts.

Java and Spring Boot provide the backend runtime and application structure. **Spring Security** controls authentication and protected operations. **Spring Data JPA** connects the application model to the database and supports record retrieval and updates.

PostgreSQL stores accounts, clubs, events, memberships and registrations. **Supabase PostgreSQL** is a compatible managed database option. In either arrangement, the Java backend remains responsible for application rules and access control.

2. Problem Statement

The problem is to provide a consistent event-management process in which published information, eligibility, remaining capacity and payment status agree across all user interfaces. The system must prevent unauthorized changes, duplicate registrations and acceptance of bookings after an event becomes unavailable.

A student should be able to identify the organizer, understand participation conditions and determine whether a registration is confirmed. An organizer should be able to manage an event without collecting the same details through several channels. Administrative review should operate on the same club and event records used by the rest of the platform.

2.1 Limitations of Existing Systems

A workflow based on separate notices, forms and payment references requires organizers to reconcile records manually. Capacity may be checked only after submissions arrive, and students may be uncertain whether submitting a form confirms a seat. General-purpose tools can support individual tasks, but college-specific approval and eligibility rules require additional coordination.

Fragmented records also make changes harder to communicate. A date may be updated in an announcement while an older form remains accessible. Participant lists can contain repeated entries, and fee information may need to be matched manually with the corresponding student.

2.2 Need for an Integrated Event and Club Platform

A shared platform connects an approved club to its published activities and links each student to a registration record. Schedule, fee and venue information is presented through the same event page. Organizers receive participant records from the booking system, reducing the need to copy data between unrelated tools.

Integration also clarifies responsibility. Clubs prepare and manage their activities, students maintain their own participation records, and administrators review organizations and moderate events. The website supports these responsibilities within one coordinated interface.

2.3 Importance of Data Privacy and User Experience

Student names, contact details, college identifiers and booking histories should be disclosed only to authorized users. Authentication alone is insufficient: the backend must also verify whether a club owns an event or a student owns a registration before allowing access to it.

The interface must explain validation errors, payment status and cancellation outcomes in language that students can act upon. Labelled fields, visible focus, readable status text and consistent navigation help users complete tasks with fewer misunderstandings. Private account details do not belong in public event listings.

2.4 Lack of Personalization and Adaptability

A single undifferentiated event list can obscure relevant opportunities. The website provides category, date and fee filters, club membership and a student dashboard of upcoming commitments. Department and year rules tailor eligibility to the selected event.

These features use explicit preferences and stated participation rules. Students can choose relevant activities without depending on automated predictions. The current application does not infer interests through machine learning or alter eligibility based on browsing behaviour.

2.5 Integration and Scalability Issues

Payment services and college identity systems introduce external dependencies. The project keeps payment behaviour behind a separate provider interface and retains application rules in the Java backend. This makes the boundary between the website and an external service clear.

Organized modules, bounded result lists and suitable database queries help the platform handle growing records. The present design is intended for a single college and a single backend instance. Wider operation would require further infrastructure and evaluation, rather than being assumed from the architecture alone.

2.6 Use Case Overview

Students discover and register for activities; clubs prepare and operate them; administrators approve organizations and moderate publication. Public browsing is available before login. Actions that change records require the appropriate authenticated role. Figure 1 summarizes these responsibilities.

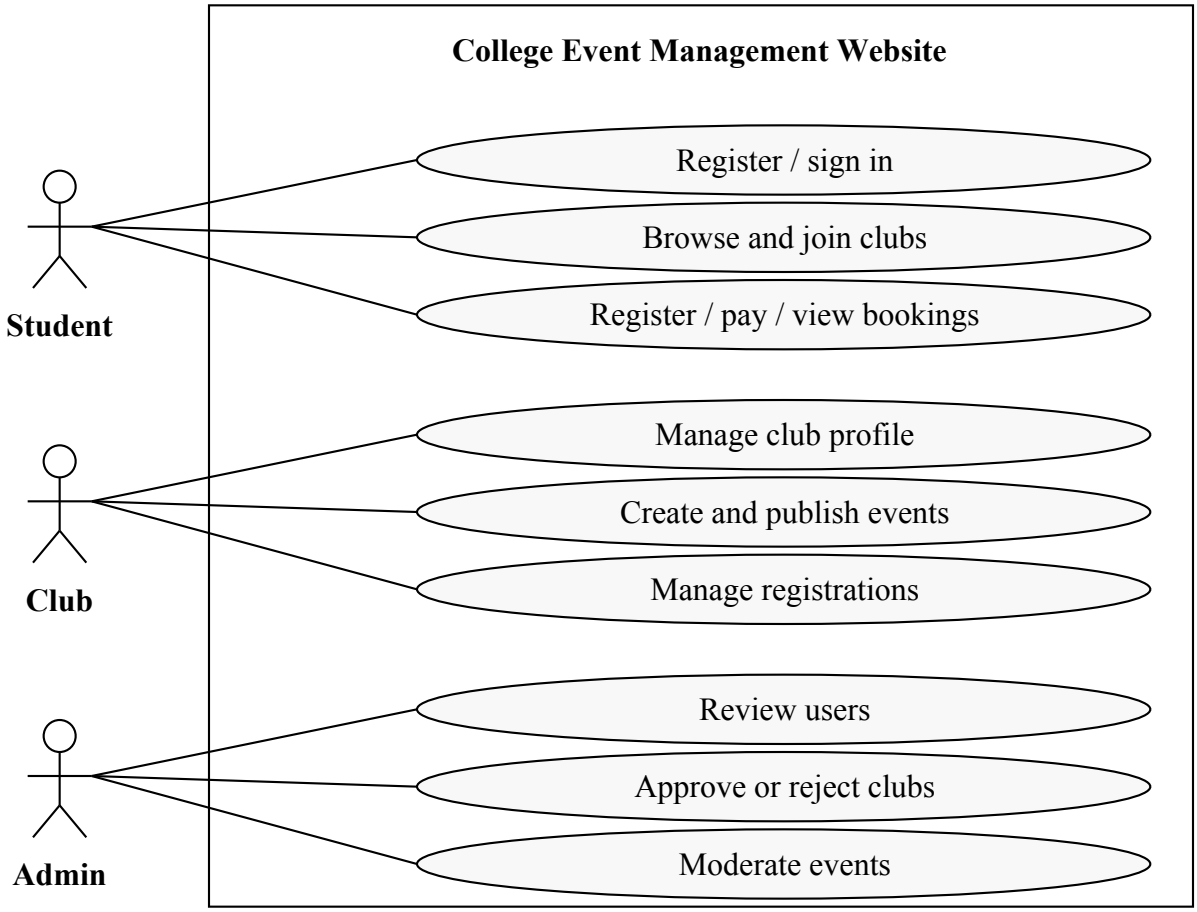


Figure 1. UML use case overview

Example Use Case Scenario

A second-year student selects a workshop and reviews its fee, date, venue and eligibility. The backend checks the profile and available capacity before creating a registration. A free event is confirmed directly; a fee-based event proceeds to the available simulated payment step. The student can review the resulting status in the registration workspace.

The organizing club sees the same registration in its participant list. An administrator can review the club's approval status and moderate the event when necessary. The scenario connects discovery, participation and administration without separate lists.

3. Literature Review

This review examines approaches relevant to event administration and the technical concepts used by the project. It compares workflow capabilities rather than ranking commercial products. The review supports the choice of an integrated website with a separate frontend, backend and database.

3.1 Evolution of Digital Event Management Systems

Online forms simplify collection, but a form alone does not inherently enforce event capacity or publication authority. Component-based interfaces and application services allow discovery, booking and management to use shared records. The React interface follows this component-based approach.

3.2 Adaptive and Personalized Event Management Models

Selected categories, joined clubs and upcoming registrations provide a transparent form of personalization. These explicit choices make relevant activities easier to locate. The project adopts this approach because its results can be understood and controlled by users.

3.3 Data Visualization and Insight Generation Systems

Event cards summarize schedule, venue, fee and availability. Workspaces place registration status and participant information near the actions that use them. This presentation helps students and clubs interpret records without reading raw database values. Detailed attendance analytics are not included in this basic model.

3.4 Integration of Security Features in Digital Applications

Access control should be applied to both the user's role and the particular record being requested. Spring Security supports the authentication boundary, while backend services check ownership and participation rules. These responsibilities complement the frontend: hiding a button alone does not protect the underlying operation.

The selected approach therefore combines an understandable interface with controlled access to shared records. Account security, appropriate disclosure and clear feedback remain part of normal application behaviour rather than separate user tasks.

3.5 Comparison to Existing Tools

Table 1 compares common organizational approaches with the proposed website. The entries describe how work is coordinated; they do not represent measured rankings of commercial products.

Table 1. Comparison of event-management approaches

Feature / Aspect	**Forms and Sheets**	**Separate Tools**	**Proposed Website**
Event discovery	Separate notices	Separate catalog	Shared catalog
Club approval	Manual review	Separate review	Admin approval
Capacity control	Manual counting	Cross-tool checks	Backend checks
Fee status	Manual matching	Separate record	Linked demo status
Data presentation	Spreadsheet rows	Separate reports	Event and booking views
Personalization	Manual selection	Separate preferences	Filters and memberships
Data privacy	File permissions	Separate accounts	Role-based access
Unified platform	Manual coordination	Multiple interfaces	Connected workspaces

3.6 Research Gaps

The main gap is between collecting registrations and maintaining a dependable process. Independent notices and forms do not automatically connect club approval, event availability, participant records and fee status.

• A shared and current source of event information is needed.

• Publication and registration must follow consistent rules.

• Personal records need restricted access and clear status feedback.

These are design requirements identified through workflow analysis. They support the proposed model without claiming a campus-wide user study or measured participation improvement.

3.7 Conceptual Framework Model

The conceptual framework connects user input to identity checks, domain operations and shared records. Each operation produces a response suited to the requesting role. Club membership, event registration and payment status remain separate concepts so that joining a club does not automatically register a student for its events.

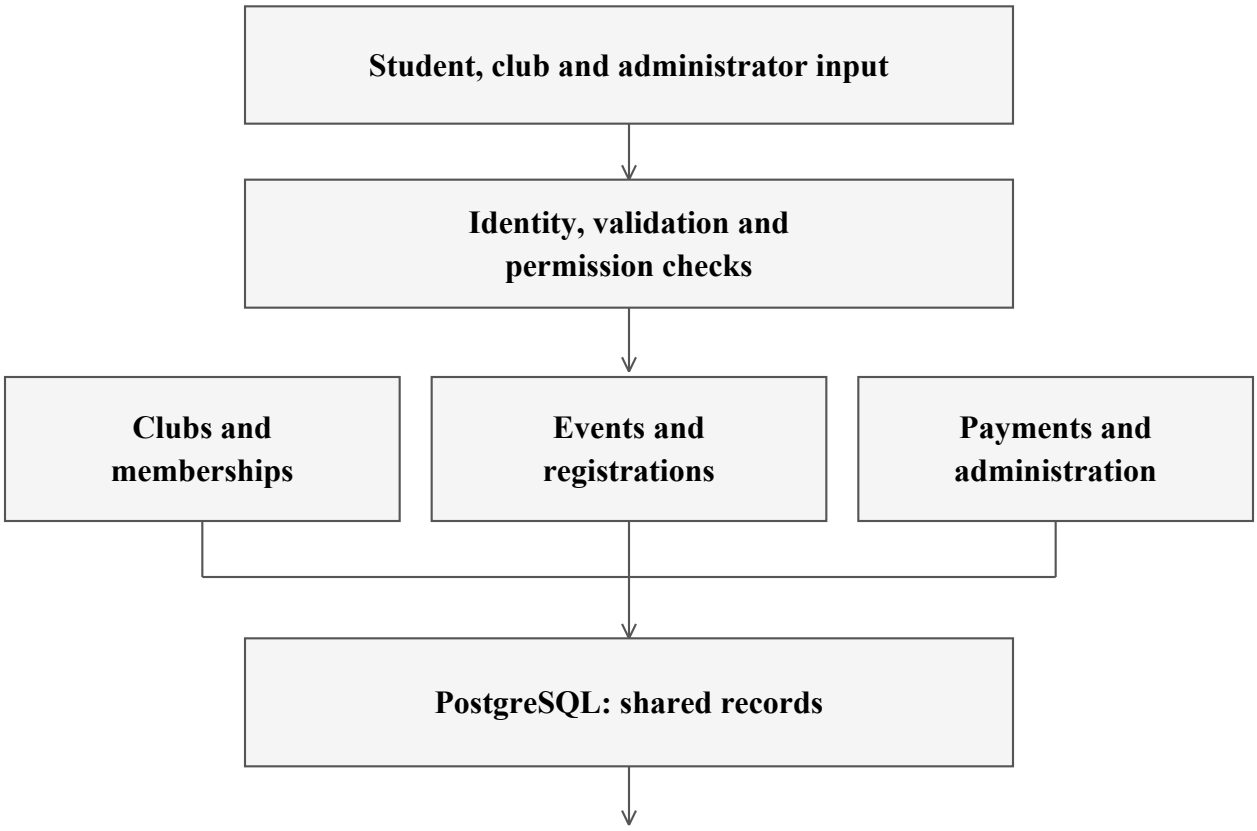


Figure 2. Conceptual framework model

Key Framework Features

Modularity: account access, clubs, events and registrations have distinct responsibilities. **Shared records:** the database provides a consistent basis for views across the platform. **User-centred interaction:** forms and status messages support the main student and organizer tasks.

Controlled access: the backend authorizes changes before saving them. **Maintainability:** the interface and external payment integration can be changed without transferring authority over event fees or capacity to the browser.

4. Proposed Model

The proposed model is a three-tier web application with public discovery and authenticated workspaces. A React single-page application communicates with a Java Spring Boot backend through JSON APIs. PostgreSQL holds the shared records. The design separates user interaction from application rules and storage.

This separation gives each layer a clear responsibility. The frontend organizes pages and collects input. The backend validates requests, identifies the permitted operation and returns the result. The database retains records needed by students, clubs and administrators.

4.1 System Overview

Students, clubs and administrators share the same catalog but receive different controls. A new club begins in a pending state and requires approval before publication. Published events accept registrations only when participation conditions are met and capacity remains available.

Students use event pages to inspect schedule, venue, fee and eligibility details. Their workspace presents joined clubs and registration history. Clubs use their workspace to maintain a profile, prepare activities and inspect participant records for their own events.

Administrators review clubs, inspect user records and moderate events. Approval affects which organizers can publish, while moderation affects public event visibility. These decisions operate on the same records used by the other workspaces.

The current payment facility demonstrates fee-related states without collecting real money. This boundary is visible in the development application. The overview therefore distinguishes the implemented registration functions from live financial settlement.

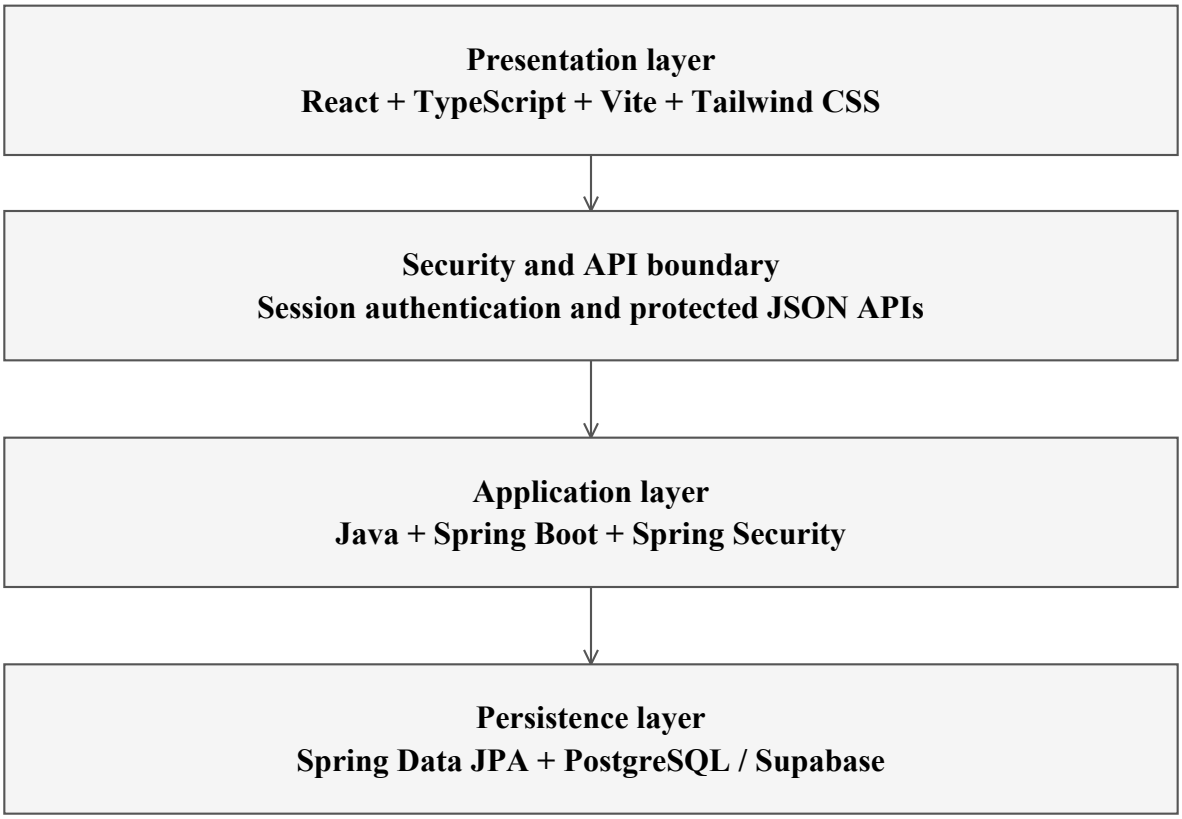
The model is intended to make event administration coherent and understandable. It combines a responsive browser interface with a Java service and a relational database, while keeping the review focused on the principal responsibilities of these components.

4.2 System Architecture

The architecture separates presentation, application processing and persistence. Security checks protect the communication between the browser and the backend. Figure 3 presents the selected technologies and their roles at a basic review level.

Architecture Components

User interface: React and TypeScript provide pages and forms; Vite and Tailwind CSS support development and styling. **Application logic:** Spring Boot processes requests and applies the platform's rules. **Storage:** Spring Data JPA manages PostgreSQL records. **Security:** Spring Security restricts authenticated operations.



The browser displays information; the backend enforces rules.

Figure 3. System architecture and technology layers

The browser sends requests to the Java service and displays the returned information. The service checks the user and requested action before accessing the database. Public event responses are separated from private registrations and administrative information.

4.3 System File Structure

The repository separates frontend code, backend code and supporting files. The following abbreviated tree retains the main implemented paths and shows how the two application layers are organized. Detailed source listings are not required for this overview.

```
Campus Commons/
    frontend/
        src/
            main.tsx, api.ts, auth.tsx, types.ts
            pages/
                Discovery.tsx, EventDetail.tsx, Clubs.tsx
                AuthPages.tsx, Student.tsx, Profile.tsx
                ClubWorkspace.tsx, EventEditor.tsx, Admin.tsx
            components/ - shared controls and layout
            styles/ - responsive presentation
    backend/
        pom.xml
        src/main/java/edu/campus/
            CampusApplication.java, SecurityConfig.java
            AuthController.java, CatalogController.java
            AdminController.java, RuntimeController.java
            AuthService.java, CatalogService.java
            RegistrationService.java, Repositories.java
            User.java, Club.java, Event.java
            Membership.java, Registration.java
            Payment.java, ResetToken.java, PaymentProvider.java
        src/main/resources/
            application.properties
            db/migration/ - database schema changes
    docs/ - product and technical documentation
    scripts/ - project commands
    README.md
```

Frontend pages correspond to student, club and administrator tasks. Backend controllers, services and data classes support the same functional areas, making their relationship easier to review.

1. Frontend Layer

The frontend handles presentation, navigation and user interaction. React pages organize the event catalog, club directory, account forms and role-specific workspaces. TypeScript models describe the information exchanged with the backend.

Reusable controls provide labelled fields, dialogs, loading states and error messages. Tailwind CSS and project styles support consistent layouts on different screen sizes. The API helper connects user actions to the corresponding backend requests.

2. Backend Layer

The backend is implemented in Java using Spring Boot. Controllers receive requests and return structured responses. Services apply rules for approval, ownership, event publication and registration. This keeps the application's decisions separate from the visual interface.

Spring Data JPA supports retrieval and updating of persistent records. Data classes represent users, clubs, events, memberships, registrations and payment information. PostgreSQL retains these records, while schema migration files document the database structure.

3. Security & Integration

Spring Security manages authenticated access. Students can inspect their own registrations, clubs can manage their own events, and administrators receive review and moderation controls. The service checks these permissions when handling a request.

The API layer connects React to Spring Boot. The frontend reports the result of an operation after the backend has processed it. This arrangement avoids relying on browser controls alone to protect records or determine event availability.

A separate payment-provider interface supports the fee-related part of registration. The current implementation uses a development simulator. PostgreSQL may be hosted locally or through Supabase, while account access and application rules remain in the Java backend.

4.4 Key System Components

The platform consists of five related functional components. Each component has a clear responsibility and shares the records needed to complete the college event-management process.

1. Identity and Profile Module

This component supports signup, login, logout and profile maintenance. Students provide basic college details used in event eligibility. The frontend collects information through labelled forms, while the backend validates it and controls access to the account.

2. Club Module

Clubs maintain their names, descriptions, categories and contact information. Students can browse approved clubs and join or leave them. Administrative approval controls publication privileges, separating organization review from ordinary student participation.

3. Event Module

Clubs create, edit and publish events with a date, venue, capacity, fee and eligibility conditions. The public interface presents these details for discovery. The backend checks ownership and publication status so that an organizer manages only its own activities.

4. Registration and Payment Module

Students register for eligible events and view the resulting status. Free events are confirmed directly. Fee-based events use the available simulated payment facility, which records the outcome without collecting money. Cancellation and registration history remain connected to the event record.

5. Dashboard and Administration Module

Student dashboards present commitments and memberships. Club workspaces present owned events and participant records. Administrative screens support user review, club approval and event moderation. Together, these views make the same underlying information accessible according to each role's responsibility.